Lecture 7

Source Code Management Control:

A source tool like git. What does it do:

Keep source code and branches, Track source code changes, Merge changes to the original code.

Key features of VCT:

Version/release tagging → when committing to the repo, we provide a commit message and sometimes we can tag it with a version number git tag v1.0.0

Tracking of change history

Independent, parallel development

Merging/conflict management

Storage efficiency (stores only deltas) —-> Saves only the differences (deltas) between file versions rather than full copies, reducing storage space.

Git Flow:

In Git we don't push directly to the main branch, we create another branch called features, and the this branch is only merged with the main branch when we finish in case we find any bugs, errors we create a branch called hot fixes from a version tag specially to fix an issue.

Git (2005):

Version control by Linus Torvalds

when you clone a repo into my local computer it also has the version history commit history and everything from the creation of this repo stored into my local machine

Git has 3 states for each file:

Modified: You've changed the file in your working directory, but Git hasn't saved those changes yet. Think of it as "work in progress."

Staged: You've marked the modified file to be included in the next commit using git add. It's like saying, "I'm ready to save this change."

Committed: The file's changes are safely stored in Git's history. This means Git has recorded a snapshot of the file in the repository.

Git Remotes:

- Git remotes are other copies of your repository, usually on another computer or server.
- Two main types:
    - Client-side → your local machine.
    - Server-side → services like GitHub, GitLab, etc.
- You sync changes between them using:
    - git pull → get updates from the remote to your local repo.
    - git push → send your changes from local to remote.
- When you clone a repo, Git calls the original repo origin by default.
- Most people only pull/push to origin.
- Because origin is usually a server, this feels like a centralized system, but Git is still distributed.
- You can have multiple remotes, and origin itself can have its own upstream.

In short: remotes are "other copies" of your repo, usually on servers, and pull/push keeps them in sync.

Git Repositories:

We have 4 main places where our code lives

Local Directory(workspace) Staging Area(index) Local Repo Remote git repo

git add -u: stages only the files that are already tracked (modified or deleted), but ignores new untracked files.

Git commit -a -m: adds and commits changed tracked files

Pull and rebase: they fetch the data and merge it into your local directory.

Fetch: downloads commits, branches, and tags from a remote repository (like origin) to your local repository, but does not change your working directory. Does not merge.

Checkout: changes the branch we are using, moves us from the staging area into the directory area.

Checkout head: cancels the uncommited changes and moves us from the local repo into the working directory.

git remote add origin github/my-repo: this adds a repo in our directory under the name github/myrepo this only exists on our local machine and will be later pushed into github as a remote repo

echo 'hello world!' > file.txt: this echo is used just for printing so we are printing hello world into the terminal and saving it into file called file.txt using a command of >

.gitignore:

That means the content of gitignore is stored on your local machine and can be pushed but should not.

Some files should not be committed to git for security and conflict issues:

Configuration files for your local setup (that may conflict with the setup of others)

Reproducible build artifacts, such as Jars, executables, generated code, ...

Libraries downloaded by your dependency management system

In git ignore we can list multiple files to put them inside it and we can use operators like * to select multiple files

Dependency Management:

Large projects use many libraries (dependencies), some of these changes in versions can break the code. Other libraries might also use other libraries which means one updated and one not may break the code.

Some libraries are only needed for development (e.g., JUnit)

We need a way to:

- Specify which libraries and which versions we use
- Ensure all team members use the same dependencies

Dependency Management Tools:

We have some softwares that can handle this dependency management issues for us.

These tools configure the correct dependency version, download dependency library and check for dependent other libraries and download them.

Dependencies are downloaded from software repositories, Usually there is one central default repository, You can configure alternative repositories or proxies, Some tools also handle building, releasing, and deploying your project

e.g for java maven, gradle. For python conda, pip. And for JS npm, yarn

Apache Maven:

It's a dependency management and build tool for java

Specifies dependencies in structured XML file:

Project Object Model XML → pom.xml

Dependencies are specified using fixed and unique versions

Transitive dependencies are resolved using each dependency's pom.xml

We have a central dependency repo called maven center

Maven tool performs its steps according to the maven lifecycle, each lifecycle step is performed by a plugin. Additional plugins can be added and configured to run in the lifecycle.

Maven Project Structure:

Project root directory → The top-level folder containing all project files and subdirectories.

Source directory → Contains the main source code of the project. Including the test assets and the main code assets, resources as well.

Java Sources → Java files that will be compiled and packaged into the final application.

Resources → Files like assets or configuration that are included in the application package.

Test Sources → Java files used only for testing and not included in the final application.

Target / Output directory → Where compiled .class files, packaged JARs, and generated code are stored.

pom.xml → The configuration file that defines the project, its dependencies, plugins, and build instructions.

Maven Declaring a Dependency:

When declaring a dependency we shall also declare the scope, when it runs.

Compile (default) – The dependency is available in **all classpaths** (compile, runtime, test). This is the default if no scope is specified.e.g: Web Framework, UI, Maths

Provided – The dependency is available at compile time but is not included in the final packaged application (because the runtime environment already provides it, e.g., a servlet API). E.g: Database connection drivers

Runtime – The dependency is not needed for compilation, but is required when running the application.

Test – The dependency is only available for testing, not included in the final package. E.g.: JUnit and other testing frameworks/tools.

System – The JAR exists on your local system; Maven does not download it from a repository, and you must explicitly specify its path. JUnit and other testing frameworks/tools

import – Only applies when importing a pom.xml as a dependency; no actual JAR is included. Dependency management in frameworks (e.g., Spring); overrides versions for transient dependencies

Maven – Remote and Local Repository:

All libraries can be downloaded from the remote repository, Usually Maven Central

Maven only downloads a dependency if is not already in the local repository

When maven downloads an independency it usually saves it on your local machine so you don't have to install it each time usually at this address: Default location: ~/.m2 any other maven project can access this file

Maven Lifecycle:

A maven lifecycle is how maven build and runs a project, what steps it takes for compiling the code, packaging it and running it. Its not about checking the correctness of a project. It about compiling classes, making jar files, running the tests of the code that you have written.

Clean(Optional) : Removes all generated output from previous builds, such as the target directory.
Example: mvn clean deletes old .class files and JARs so you start fresh.

Validate: Checks that the project structure is correct and all required information is available.

Compile: Compiles the main source code of the project into .class files.
Example: mvn compile turns src/main/java/App.java into target/classes/App.class.

Test : Runs unit tests on the compiled code using a testing framework like JUnit

Package: Takes the compiled code and packages it into a distributable format like JAR, WAR, or ZIP.

Verify: Runs additional checks on the packaged application to ensure it meets quality standards.

Install: Copies the built package into the local Maven repository (~/.m2) for use by other local projects.

Deploy: Uploads the final package to a remote repository so it can be shared with other developers or projects.

The lifecycle is about building and managing the project you are working on, not the dependencies themselves (though dependencies are fetched as part of some phases, like validate or compile).

Maven Command line interface:

We can run maven lifecycle and specify certain phases from the lifecycle to be executed.

When you rung mvn clean package → maven cleans your certain package.

When you run a certain phase in maven command line it does not only run this certain phase but also all the phases before it.

Multiple lifecycle-steps can be provided, Mostly used to also specify otherwise optional steps to be executed (almost always: clean)does this here mean that you can run anything but we mostly run clean

N.B: you can run multiple command while using mvn for instance mvn clean test package. This cleans the old files, then runs all the phases before test including test phase on the project/package you target.

Maven Plugins:

Maven Surefire Plugin:

Executes Unit Tests

its the plugin used by mvn by default to run the test phase in the maven lifecycle

Maven JAR Plugin:

Packages a JAR

Default Plugin for package step

Maven Javadoc Plugin:

Automatically generates Javadoc documentation

Can be configured to run right before compile (generate-sources step)

Maven Shade Plugin:

Packs library dependencies into “fat” JAR, a JAR is a java archive file that stores all the compiled source code without dependencies, the fat JAR is a file done by the plugin to include the dependencies.

Can be configured to run during package step

Dependency Management Systems – Fixed Versions vs. Version Ranges:

Maven and Gradle only support fixed and unique versions.

npm (Javascript), pip (Python), gem (Ruby), and others support version ranges.

Those libraries allow automatic version updates and version ranges using a format

Major.Minor.Patches → Major never changes, minor changes when we include ^, patch always changes

e.g: ~32.0.1 → this means the major stays fixed, the minor also stays fixed but the patch should change. This means that this can be 32.0.3 only the last digit can change.

e.g: ^8.2.2 → this means that both the minor and the patch can change. This means that the version can be 8.2.2, 8.3.2, 8.4.2 and so on the minor changes can only increase and the patch of course

In case we did not include ~ or ^ the version stays the same then its fixed.

Maven Snapshot Version:

The groupid is for the developing entity of the dependency or the package you are using

The artifactid is the name of the dependency we are using.

The version snapshot indicates that the version of the repo we are using is not the final one and can be further updated. It’s the way maven allows dependency updates automatically.

To automatically update all the snapshot libraries we can run on the CLI mvn install -U this force updates all the dependencies

Final versions makes maven believe that the repo is final and the repo never allows updates

Advantages and disadvantages of version ranges?:

Advantages of version ranges:

Automatic minor/patch updates

Less maintenance: You don't have to update the version number every time a compatible new version of a dependency is released.

Safety with Semantic Versioning: Assuming libraries follow SemVer rules, updates with ^ or ~ should not break your code (minor/patch updates are backward-compatible).

disadvantages of version ranges:

Potential hidden breakages: If a library doesn't strictly follow SemVer, minor/patch updates can introduce breaking changes, causing your code to fail unexpectedly.

Harder debugging: When errors occur, it can be tricky to figure out which exact dependency version caused the issue if versions are automatically updated

Version ranges can lead to dependencies breaking: when you auto update a dependency it may include changes in the code syntax for instance this change can lead to code breakage. Which lead to someone saying "it works on my machine"

Lockfile:

A lockfile is a file that includes meta data about your project dependency, it stores which dependency is on which version.

This file is read then by your dependency manager each time you update dependencies, it makes sure that every one is using the same dependencies versions'.

It very important that you commit this lockfile to your git repo

Testing on Git Push – Basic Idea:

the main idea is to build a pipeline while pushing to git, since the CI/CD pipeline will run your already written unit tests automatically each time and check for results then notify you

Why is testing on a build server after every Git push a good idea?:

Early bug detection: when you run the tests automatically each time you push your code you make sure that your code issues does not accumulate.

Consistency across environments: everyone has the same code, everyone can see the same issues, same tests.

Continous feedback: developers notified instantly when there is a problem found

Team collaboration and automation; the entire team sees the same code, tests, defects at the same time, it easier then to be fixed by the entire team. Automated tests save time and repetitive tasks.